

CIS 2109 Exam 1 Study Guide :)

Hi guys! I made this study guide to touch on material I believe will be on the exam. **I don't exactly know the specific material or what's been covered so far in the lectures, so please take everything in here with a grain of salt.** I've included some notes about relations, ER diagrams, and also some explanations about well-structured database design.

What is a Relation?

A named, two-dimensional table of data that describes one and only one topic.

- Consists of rows (records/instances) and columns (attributes/fields)

For a table to qualify as a relation, all attribute values must be atomic — no multi-valued or repeating groups are allowed.

Keys

Key Type	Definition & Role
Primary Key (PK)	Uniquely identifies every row in a relation. <u>Cannot be NULL</u> . Guarantees row uniqueness. ★ Can be simple (one column) or composite (multiple columns).
Foreign Key (FK)	An attribute in a dependent (child) relation that references the PK of a parent relation. Enforces referential integrity. ★ Can be self-referencing. A table can have multiple FKs.
Composite Key	A PK made of two or more columns; used when no single column is unique on its own (e.g., StudentID + CourseID in an enrollment table).

Update Anomalies

Anomalies occur when a database is poorly designed and data is not properly separated into distinct relations.

There are three types:

Anomaly Type	Description & Example
Insertion Anomaly	The table structure prevents you from adding data without also adding unrelated data. E.g., you cannot add a new department until at least one employee is assigned to it.
Deletion Anomaly	Deleting one record unintentionally destroys other needed data. E.g., deleting the last employee in a department also deletes the department's information.
Modification Anomaly	Changing one fact requires updating many rows. E.g., if a department name is stored in every employee row, renaming the department requires updating every employee record.

Integrity Constraints

Constraint	Rule
Entity Integrity	Every relation must have a PK. PK values must be unique and NOT NULL.
Referential Integrity	FK values must match an existing PK in the parent table, or be NULL (if optional). Prevents orphan records.

SECTION 2 — Business Rules & ER Diagram Design

Business Rules

Business rules are statements that define or constrain some aspect of the business. They are gathered from stakeholders (interviews, policy documents) and form the foundation of the data model. A good business rule is:

- Declarative — states what, not how
- Atomic — expresses one idea
- Precise — unambiguous
- Consistent — does not contradict other rules
- Distinct — not redundant

Naming Standards for Data Objects

Rule	Detail
Related to the business	Names must be meaningful, self-documenting, and drawn from an approved vocabulary.
Unique	No two data objects share the same name.
Singular or Plural (consistent)	All entity names follow the same convention (e.g., all singular: Customer, Order, Product).
Good examples	Customer, Order, Product, Employee
Bad examples	File10, Bit7 (non-meaningful); Has, Is, Person (verbs or too generic)

ER Diagram Components

Component	Description
Strong Entity	Exists independently; has its own unique identifier (PK). Drawn as a single rectangle.
Weak Entity	Depends on a strong (owner) entity for its existence and identity. No PK of its own — only a partial key. Drawn with a double rectangle.
Identifying Relationship	Links a strong entity to its weak entity. Drawn with double lines..
Simple Attribute	Single, atomic value (e.g., LastName).
Composite Attribute	Made of sub-attributes (e.g., Address {Street, City, State, ZIP}).
Identifier Attribute	Underlined — uniquely identifies instances of the entity.
Unary Relationship	An entity related to itself (e.g., Employee supervises Employee). Also called a recursive or self-referencing relationship.

Cardinality & Participation

Notation	Meaning
1:1 (one to one)	One instance of A is associated with exactly one instance of B.
1:N (one to many)	One instance of A is associated with many instances of B.
M:N (Many to Many)	Many instances of A are associated with many instances of B. Should be modeled as an associative entity to properly track data.
Total Participation (known as mandatory)	Every instance of the entity MUST participate in the relationship. Marked with a single line in addition to the one or many symbol.
Partial Participation (known as optional)	Some instances of the entity may not participate in the relationship. Marked with a circle in addition to the one or many symbol.

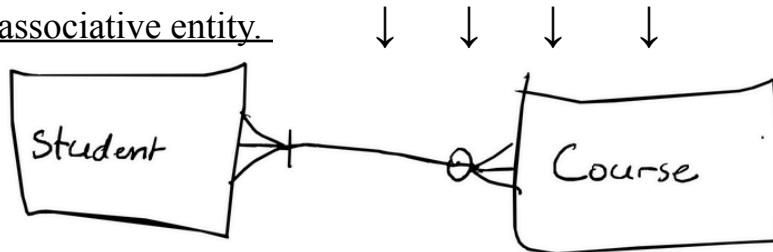
Associative Entities

An associative entity (also called a bridge or junction entity) is used when:

- All relationships in the M:N are 'many' on both sides
- The association itself has independent meaning — i.e., the relationship has its own attributes (e.g., Grade in an Enrollment entity, Quantity in an OrderLine entity)

An associative entity is typically assigned a composite PK made of the PKs of both participating entities, or a new surrogate PK.

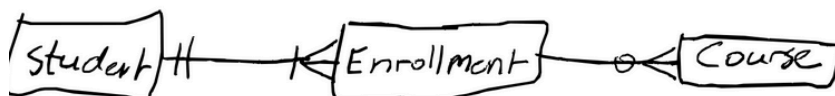
Basically, if you see a many to many relationship and need to track attributes, model it as an associative entity.



"Student may enroll in many courses"

"Course must be taken by at least one student"

Doesn't tell us much about which student is actively enrolled in a course



Enrollment entity now lets us track which student is enrolled in which course and maintains integrity

Time-Dependent Data

When an attribute's value changes over time and history must be preserved (e.g., the price of a product), a simple attribute is insufficient. Solutions include:

- Adding a timestamp/date attribute to record when the value was current
- Creating a separate history table (e.g., Price_History with ProdID, Price, EffectiveDate)
- Modeling the price as part of an associative entity (e.g., OrderLine with Price_Per_Item at time of order)

SECTION 3 — Extended ER (EER) Diagrams

Supertypes & Subtypes

EER diagrams extend basic ER notation to handle inheritance. A supertype is a generic entity with attributes shared by all subtypes. A subtype is a specialized version of the supertype and inherits all of its supertype's attributes.

Concept	Definition
Supertype	Generic entity that groups shared attributes of related subtypes. Every subtype instance is also a supertype instance → (For example, every cat (subtype) is still an example of an animal (supertype))
Subtype	Specialized entity inheriting all supertype attributes, plus having its own unique attributes. → (A cat is a subtype of animal that has unique features such as quick reflexes and sharp claws)
Attribute Inheritance	A subtype automatically has all attributes defined on the supertype. (A cat, being a type of animal, has all of the features that make up a basic animal. According to Google, that's their ability to move spontaneously, ingest food (heterotrophy), and develop from a blastula).
Generalization	Bottom-up process — recognize shared attributes across several entities and create a common supertype.
Specialization	Top-down process — recognize that one entity has attributes that only apply in certain cases; split into subtypes. - Useful especially for if certain attributes only apply to specific conditions

EER Constraint Rules

Constraint	Notation
Total Specialization	Double line between supertype and circle — every supertype instance MUST belong to at least one subtype. (the system makes you say what kind of employee you are (manager, clerk, etc) and won't let you just say 'employee')
Partial Specialization	Single line — a supertype instance may or may not belong to any subtype (if the only options in the system for employees are either manager or employee, then an intern for example can just exist as a generic 'employee')
Disjoint Rule (d)	'd' inside the circle — a supertype instance can belong to ONLY ONE subtype. Mutually exclusive. For example, an animal can be a dog or a cat, but not both. (what would that even look like?)
Overlap Rule (o)	'o' inside the circle — a supertype instance can belong to MORE THAN ONE subtype simultaneously. For example, at a university you can be a student, an employee, or both (like me!)

Subtype Discriminator	An attribute on the supertype that indicates which subtype(s) an instance belongs to. Read more below.
-----------------------	--

On Discriminators

Discriminators will look different depending on whether or not it is disjoint or overlap. For disjoint subtypes, you can simply include a single attribute to distinguish this (e.g., AccountType = 'Checking'/'Savings'/'Loan'). For overlapping subtypes, you need to introduce attributes for each subtype, usually in the form of a true/false boolean. (for example a spork can be modelled as such: isSpoon=True, isFork=True, isKnife=False)

EER Example — Account

Supertype: ACCOUNT (AcctNo, DateOpened, Balance, AcctType)

Subtypes: LOAN (InterestRate, PaymentSchedule), CHECKING (ServiceCharge), SAVINGS (InterestRate)

Constraint: Partial + Disjoint — an account may be only one type (checking, savings, or loan), and some accounts may not yet be classified.

SECTION 4 — Relational Schema Mapping

Mapping Rules Summary

ER Construct	Mapping Rule
Strong Entity	One table. PK = key attribute of the entity.
Weak Entity	One table. PK = composite of partial key + FK of owner entity. FK must reference the owner's PK.
1:N Relationship	No new table. Add the PK of the ' one ' side as a FK on the ' many ' side.
M:N Relationship	<u>Create an associative entity</u> . The associative entity's PK becomes a composite of both entities' PKs. Both also become FKs.
1:1 Relationship	Add FK on the mandatory side. If both are optional/mandatory, pick whichever side makes more sense.
Multivalued Attribute	Create a separate table with a FK back to the owning entity. PK = (EntityPK + the attribute value). This can be modeled as a dependent entity.
Composite Attribute	Store each sub-attribute as its own column in the entity's table. For example, if a Person entity has Name{FirstName, MiddleName, LastName} as an attribute just add FirstName, MiddleName, Lastname as separate attributes.
Associative Entity (ID assigned)	You can choose to assign an associative entity its own ID.
Associative Entity (no ID)	If you don't assign an associative entity its own ID, its primary key becomes a combination of both foreign keys.
Supertype/Subtype	One table for the supertype (all shared attrs + discriminator). One table per subtype (subtype-specific attrs + FK to supertype PK).
Unary 1:N	One table with a self-referencing FK (e.g., ManagerID references EmpID in the same EMPLOYEE table).
Unary M:N	One table + a junction table that has two FKs both referencing the same entity's PK. (An associative entity with a composite primary key)

Mandatory vs. Optional FK Constraints

Specify if an attribute is mandatory by adding NOT NULL to the field

Normalization / Normal Forms (More explained on the next page)

Category	Commands & Purpose
First Normal Form (1NF)	Removes repeating groups and ensures every column contains atomic values (every column describes only one thing)
Second Normal Form (2NF)	Must be in 1NF and also make sure all non-key attributes are dependent on the entire primary key
Third Normal Form (3NF)	Must be in 2NF and eliminate transitive dependencies. Transitive dependencies occur when non-key attributes depend on other non-key attributes

Let's say we have the following table.

Employee

EmpID	Name	Skills	DeptCode	Dept
101	Bingus	goofing off, bantering	1	Executive
102	Schmidley	lollygagging, naysaying	1	Executive

This table sucks because it is very unoptimized. To start, the skills column describes multiple items. So, we split this such that each entry only describes one item ↓

Employee

EmpID	Name	Skill	DeptCode	Department
101	Bingus	goofing off	1	Executive
101	Bingus	bantering	1	Executive
102	Schmidley	lollygagging	1	Executive
102	Schmidley	naysaying	1	Executive

This is 1NF.

Let's assume the primary key is composite between EmpID and Skill. The problem now is that Name, DeptCode, and Department only rely on EmpID and not the full composite key. This is called a partial dependency. To fix this, we split it into two tables.

Employee

EmpID	Name	DeptCode	Department
101	Bingus	1	Executive
102	Schmidley	1	Executive

This is 2NF. For a schema to be in 2NF, it must first be in 1NF. Now, the issue is that Department relies on DeptCode, which in turn relies on EmpID to enforce uniqueness (Name doesn't count, as it's not a primary key). This is called a transitive dependency.

To fix this, we again split into separate tables:

Employee

EmpID	Name	DeptCode
101	Bingus	1
102	Schmidley	1

Departments

DeptCode	Department
1	Executive

EmployeeSkill

EmpID	Skill
101	goofing off
101	bantering
102	lollygagging
102	naysaying

This is now 3NF.

SECTION 5 — Oracle SQL: DDL

SQL Language Categories

Category	Commands & Purpose
DDL — Data Definition Language	CREATE TABLE, ALTER TABLE, DROP TABLE, TRUNCATE — defines and modifies database structures.
DML — Data Manipulation Language	INSERT, UPDATE, DELETE, SELECT — adds, changes, removes, and retrieves data.
DCL — Data Control Language	GRANT, REVOKE — controls access permissions. All DDL statements imply an automatic COMMIT.

Oracle Data Types

Type	Use
NUMBER(p, s)	Numeric values. p = total digits, s = decimal scale. E.g., NUMBER(8,2) for currency.
VARCHAR2(n)	Variable-length string up to n chars. Preferred over CHAR for most text fields.
CHAR(n)	Fixed-length string, always n chars (padded with spaces). Use for codes like state abbreviations.
DATE	Stores date AND time (year, month, day, hour, minute, second). Needs TO_DATE() for string conversions.
INTEGER / NUMBER(38)	Whole numbers with no decimal component.

CREATE TABLE — Full Example

```
CREATE TABLE STUDENT (  
  StudentID  NUMBER(8)   PRIMARY KEY,  
  LastName   VARCHAR2(50) NOT NULL,  
  FirstName  VARCHAR2(50) NOT NULL,  
  Major      VARCHAR2(30),  
  GPA        NUMBER(3,2) CHECK (GPA BETWEEN 0.0 AND 4.0),  
  DOB        DATE,  
  DeptID     NUMBER(5)   REFERENCES DEPARTMENT(DeptID)  
              ON DELETE SET NULL  
);
```

ALTER TABLE

```
-- Add a column  
ALTER TABLE STUDENT ADD (PhoneNumber VARCHAR2(15));  
  
-- Modify a column size  
ALTER TABLE STUDENT MODIFY (LastName VARCHAR2(75));  
  
-- Add a named constraint  
ALTER TABLE STUDENT ADD CONSTRAINT chk_gpa CHECK (GPA >= 0);
```



```
-- Drop a constraint
ALTER TABLE STUDENT DROP CONSTRAINT chk_gpa;
```

DROP / TRUNCATE

```
-- Remove table entirely (fails if FK references exist)
DROP TABLE STUDENT;

-- Force drop and remove dependent FK constraints
DROP TABLE STUDENT CASCADE CONSTRAINTS;

-- Remove all rows but keep the table structure
TRUNCATE TABLE STUDENT;
```

*****I am not sure if you guys have covered up to the following materials in the lectures yet or if it will be in the exam. If you haven't skip to the end for practice questions**

SELECT Statement Structure

The SELECT statement retrieves data. Clauses must appear in this order:

Clause	Purpose
SELECT	List columns to return. Use * for all columns. Use column aliases with AS.
FROM	Specify the table(s) to query.
WHERE	Filter rows based on a condition. Evaluated before grouping.
GROUP BY	Group rows by one or more columns for aggregation.
HAVING	Filter groups (used with GROUP BY). Like WHERE but for groups.
ORDER BY	Sort the result set. Always the LAST clause.

Key SELECT Features & Oracle Functions

Feature / Function	Syntax & Notes
Column alias	SELECT Salary * 12 AS AnnualSalary FROM EMPLOYEE;
MAX() / SUM()	SELECT MAX(Salary), SUM(Salary) FROM EMPLOYEE;
Exact match WHERE	WHERE LastName = 'Smith' — strings must be in single quotes, case-sensitive.
Comparison WHERE	WHERE LastName > 'M' — alphabetical comparison.
LIKE — wildcard	WHERE Name LIKE 'Sm%' → starts with 'Sm'. '%son%' → contains 'son'. NOT LIKE to exclude.
IS NULL / IS NOT NULL	WHERE ManagerID IS NULL — cannot use = NULL in Oracle.
NVL(col, 'replacement')	Replaces NULL values with the given default for display purposes.
LOWER() / UPPER()	LOWER(LastName) converts to lowercase; UPPER() to uppercase.
INITCAP()	Capitalizes the first letter of each word.
Concatenation	LastName ', ' FirstName — or CONCAT(col1, col2) for two inputs.

INSTR(col, 'str')	Returns position of first occurrence of 'str' in col.
RTRIM() / LTRIM()	Removes trailing / leading whitespace from a string.
ROUND(n, pos)	Rounds n. pos=0 → ones place, pos=1 → tenths, pos=-1 → tens.
TO_DATE()	Converts a string to a DATE. E.g., TO_DATE('03/19/2005','MM/DD/YYYY').
TO_CHAR(date,'YYYY')	Converts a date to a formatted string for display.
EXTRACT(YEAR FROM date)	Extracts the year component from a date. E.g., EXTRACT(YEAR FROM SYSDATE).

DELETE Statement

```
DELETE FROM EMPLOYEE WHERE DeptID = 10;
-- Removes all rows where DeptID = 10.
-- Always use a WHERE clause unless you intend to delete all rows.
```

Transactions in Oracle

A transaction is a set of SQL statements that Oracle treats as a single logical unit. DDL statements (CREATE, DROP, ALTER) automatically COMMIT. DML statements (INSERT, UPDATE, DELETE) do NOT auto-commit — you must issue a COMMIT or ROLLBACK manually to make changes permanent or undo them.

See questions on next page

Practice Questions

1. A product's price changes over time and you must track the historical prices. Describe two ways to model this and explain the trade-off.

Answer: _____

2. A student can be enrolled in many courses, and a course can have many students. The enrollment has a Grade and a Semester. Should Grade and Semester be modeled as attributes of STUDENT, COURSE, or as an associative entity? Justify your answer.

Answer: _____

3. What is a unary relationship? Model the following: 'An employee may supervise other employees.' Describe the entities and relationship including cardinality.

Answer: _____

4. A table that stores a comma-separated list of phone numbers in a single column — e.g., '215-555-0101, 610-555-0189' — qualifies as a valid relation.

Answer: [True / False]

5. Defining a PRIMARY KEY automatically enforces both uniqueness and NOT NULL on that column — no additional constraint is needed.

Answer: [True / False]

6. A foreign key column must always be NOT NULL.

Answer: [True / False]

7. Two rows in a relation can share the same primary key value as long as their other column values are different.

Answer: [True / False]

8. Placing a 'Grade' attribute directly on the STUDENT entity is correct when STUDENT participates in an M:N relationship with COURSE.

Answer: [True / False]

9. Which of the following would violate referential integrity?

- A) A NULL value in an optional foreign key column.
- B) Two rows sharing the same foreign key value.
- C) A foreign key value of 50 when no row with PK = 50 exists in the parent table.
- D) A child table having more rows than the parent table.

10. A table stores: OrderID, CustomerID, CustomerName, and CustomerCity. OrderID is the PK. A developer updates CustomerCity in one row. The same customer's city is now different across 12 other rows. Which anomaly does this describe?

- A) Insertion anomaly
- B) Deletion anomaly
- C) Modification anomaly
- D) Domain constraint violation

11. Which of the following is the best definition of entity integrity?

- A) Every foreign key must reference an existing primary key.
- B) Every table must have a primary key, and its values must be unique and never NULL.
- C) Every column must have a value — NULLs are not permitted anywhere.
- D) Every row must satisfy all CHECK constraints defined on the table.

12. A designer is modeling 'An employee may manage other employees.' Which ER construct is needed?
- A) Two separate EMPLOYEE entities connected by a 1:N relationship.
 - B) A unary (recursive) relationship on EMPLOYEE, with a self-referencing FK in the schema.
 - C) An associative entity between EMPLOYEE and MANAGER.
 - D) A supertype EMPLOYEE with subtype MANAGER.
13. When should a M:N relationship be converted into an associative entity rather than left as a plain diamond?
- A) Always — every M:N must be an associative entity.
 - B) Only when both sides have total participation.
 - C) When the relationship has its own attributes, or has independent meaning beyond linking two entities.
 - D) Only when the relationship is unary.
14. An ACCOUNT can be a CHECKING account, a SAVINGS account, or a LOAN — but never more than one at the same time. Every account must be one of the three. How would this be modeled?
- A) Partial specialization + Overlap
 - B) Total specialization + Overlap
 - C) Partial specialization + Disjoint
 - D) Total specialization + Disjoint
15. At a university, a PERSON can be a STUDENT, a FACULTY member, or both simultaneously. Some people on campus (e.g., administrative staff) are neither. Which EER constraints apply?
- A) Total specialization + Disjoint
 - B) Total specialization + Overlap
 - C) Partial specialization + Disjoint
 - D) Partial specialization + Overlap

Schema:

STUDENT (StudentID, LastName, FirstName, Email, MajorName, MajorBuilding)

COURSE (CourseID, CourseName, Credits, DeptCode, **ProfessorID**)

PROFESSOR (ProfessorID, ProfName, **DeptCode**)

SECTION (SectionID, , Semester, Year, Enrollment)

ENROLLS (StudentID, CourseID, Grade, EnrollDate)

underline = Primary Key **Bold** = Foreign Key

(foreign keys are supposed to be dashed underlines but i couldn't get it working in the docs my fault y'all)

16. What type of relationship does the ENROLLS table resolve? How can you tell from its structure?
17. A professor is hired but has not yet been assigned to teach any course. Can this professor be inserted into the database using the current schema? Does an anomaly exist?
-
18. ProfessorID in COURSE is a foreign key. What does this enforce? What happens if someone tries to insert a COURSE row with a ProfessorID that doesn't exist in the PROFESSOR table?
-

Schema:

PATIENT (MRN, PatientName, DOB, **PhysicianID**)

PHYSICIAN (PhysicianID, PhysicianName, Specialty)

WARD (WardID, WardName, **HeadNurseID**)

NURSE (NurseID, NurseName, **WardID** NOT NULL)

ADMISSION (AdmissionID, **MRN**, **WardID**, AdmitDate, DischargeDate)

underline = Primary Key **bold** = Foreign Key NOT NULL where shown

19. NURSE.WardID is marked NOT NULL. What does this tell you about the relationship between NURSE and WARD?

20. PATIENT.PhysicianID is a foreign key with no NOT NULL constraint shown. What does this mean for how patients are stored?

21. Can the ADMISSION table record the same patient being admitted more than once over time? How does the schema support or prevent this?

Schema:

ORDER_RECORD (OrderID, CustomerID, CustomerName, CustomerEmail,
ProductID, ProductName, ProductCategory,
Quantity, UnitPrice, OrderDate)

underline = Primary Key (This is a single unnormalized table — no FKs defined)

22. This table is not in 2NF. Identify all partial dependencies present. (Assume the PK is OrderID alone)

23. Which anomaly would occur if you tried to add a new product to the catalog before any orders have been placed for it?

- A) Deletion anomaly
- B) Modification anomaly
- C) Insertion anomaly
- D) Referential integrity violation

24. Normalize ORDER_RECORD into a set of 3NF-compliant tables. List each table with its columns and mark the PK of each.

Schema:

DEPARTMENT (DeptID, DeptName, **ManagerID** NULL)
EMPLOYEE (EmpID, EmpName, **DeptID** NOT NULL, Salary, **ManagerID** NULL)
PROJECT (ProjectID, ProjectName, **DeptID** NOT NULL)
WORKS_ON ((EmpID, ProjectID), HoursPerWeek)

EMPLOYEE.DeptID references DEPARTMENT(DeptID)
EMPLOYEE.ManagerID references EMPLOYEE(EmpID) (self-referencing, nullable)
PROJECT.DeptID references DEPARTMENT(DeptID)
WORKS_ON.EmpID references EMPLOYEE(EmpID)
WORKS_ON.ProjectID references PROJECT(ProjectID)
DEPARTMENT.ManagerID references EMPLOYEE(EmpID) (nullable)

25. Can this INSERT succeed? INSERT INTO EMPLOYEE VALUES (201, 'Jordan Lee', 99, 75000, NULL); Note: DeptID 99 does not exist in the DEPARTMENT table.

- A) Yes — the insert succeeds.
- B) No — violates referential integrity (DeptID FK). DeptID is NOT NULL and must match an existing DEPARTMENT row.
- C) No — violates entity integrity (EmpID 201 is NULL).
- D) No — violates a CHECK constraint on Salary.

26. Can this INSERT succeed? INSERT INTO EMPLOYEE VALUES (202, 'Alex Kim', NULL, 68000, NULL); Note: DeptID is NULL.

- A) Yes — DeptID allows NULL so this is fine.
- B) No — DeptID is defined as NOT NULL in the EMPLOYEE table.
- C) No — EmpID 202 duplicates an existing row.
- D) No — Salary cannot be 68000.

27. Can this INSERT succeed? INSERT INTO DEPARTMENT VALUES (10, 'Engineering', 202); — Note: No employee with EmpID 202 exists yet in EMPLOYEE.

- A) Yes — ManagerID is nullable so NULL-like behavior applies.
- B) No — violates referential integrity. ManagerID references EMPLOYEE(EmpID) and EmpID 202 does not exist.
- C) No — violates entity integrity on DeptID.
- D) Yes — DEPARTMENT has no constraints preventing this.

28. Can this INSERT succeed? INSERT INTO WORKS_ON VALUES (201, 50, 40); — Note: EmpID 201 exists. Additionally, ProjectID 50 does NOT exist in PROJECT.

- A) Yes — WORKS_ON only needs EmpID to be valid.
- B) No — violates referential integrity on ProjectID FK.
- C) No — violates entity integrity (composite PK is NULL).
- D) Yes — junction tables do not enforce FK constraints.

29. Can this INSERT succeed? INSERT INTO WORKS_ON VALUES (201, 30, 40); — A row (EmpID=201, ProjectID=30) already exists in WORKS_ON.

- A) Yes — duplicate rows are allowed in junction tables.
- B) No — violates entity integrity. The composite PK (EmpID, ProjectID) must be unique; this combination already exists.
- C) No — violates referential integrity.
- D) Yes — only the PK columns matter for uniqueness, and EmpID alone is the PK.

Again, i don't know if you guys have covered up to this material in the lectures yet or if it will be on the exam. Still good SQL practice tho

30. What does this CREATE TABLE statement define? Identify every constraint created and explain what each one enforces.

SQL:

```
CREATE TABLE ENROLLMENT (  
  StudentID NUMBER(8) NOT NULL,  
  CourseID NUMBER(5) NOT NULL,  
  Semester VARCHAR2(10) NOT NULL,  
  Grade CHAR(2) CHECK (Grade IN ('A','B','C','D','F','W','I')),  
  EnrollDate DATE DEFAULT SYSDATE,  
  CONSTRAINT pk_enroll PRIMARY KEY (StudentID, CourseID, Semester),  
  CONSTRAINT fk_student FOREIGN KEY (StudentID)  
    REFERENCES STUDENT(StudentID)  
  CONSTRAINT fk_course FOREIGN KEY (CourseID)  
    REFERENCES COURSE(CourseID)  
);
```

31. What does this ALTER TABLE statement do? What would happen if an existing row in EMPLOYEE already had Salary = 0 when this runs?

SQL:

```
ALTER TABLE EMPLOYEE  
  ADD CONSTRAINT chk_salary  
  CHECK (Salary > 0 AND Salary < 500000);
```

32. What does this query return? Walk through each clause and describe what it does.

SQL:

```
SELECT D.DeptName,  
       COUNT(E.EmpID) AS NumEmployees,  
       ROUND(AVG(E.Salary),2) AS AvgSalary  
FROM DEPARTMENT D, EMPLOYEE E  
WHERE D.DeptID = E.DeptID  
      AND E.Salary > 50000  
GROUP BY D.DeptName  
HAVING COUNT(E.EmpID) > 3  
ORDER BY AvgSalary DESC;
```

33. This query is supposed to find all employees with no manager. A developer writes it as shown below. Will it work? If not, why not — and how should it be written?

SQL:
SELECT EmpName
FROM EMPLOYEE
WHERE ManagerID = NULL;

34. What does this DDL block accomplish? Describe the end state of the STUDENT table after all three statements execute.

SQL:
ALTER TABLE STUDENT ADD (AdvisorID NUMBER(8));

ALTER TABLE STUDENT
ADD CONSTRAINT fk_advisor
FOREIGN KEY (AdvisorID) REFERENCES PROFESSOR(ProfessorID)
ON DELETE SET NULL;

ALTER TABLE STUDENT
ADD CONSTRAINT chk_email
CHECK (Email LIKE '%@%');

For business rules / diagram designs, try to complete each problem in ~15 minutes. I think this is good pacing for exams

35. University Course Registration — Draw the complete ER diagram for the following business rules.

Business Rules:

- The university tracks Students, Courses, and Professors.
- Each Student has a StudentID, FirstName, LastName, Email, and a declared Major.
- Each Course has a CourseID, CourseName, and Credits.
- A Student can enroll in many Courses; a Course can have many Students enrolled.
- When a Student enrolls in a Course, a Grade and a Semester are recorded for that enrollment.
- Each Course is taught by exactly one Professor. A Professor may teach many Courses.
- Every Course must have a Professor assigned (total participation on COURSE side).
- Each Professor has a ProfessorID, Name, and may have multiple phone numbers.

Draw ER diagram below:

36. Hospital System — Draw the complete ER diagram for the following business rules.

Business Rules:

- The hospital tracks Patients, Nurses, Physicians, and Wards.
- Each Patient has a MRN, Name, and Date of Birth. Patient Age is not stored directly.
- Each Ward has a WardID, WardName, and Capacity.
- A Ward contains many Beds. Each Bed has a BedNumber and belongs to exactly one Ward. A Bed cannot be identified without knowing its Ward.
- A Patient may be admitted to a Bed multiple times over their history. Each admission records an AdmissionDate and DischargeDate.
- Each Nurse must be assigned to exactly one Ward (mandatory). A Ward must have at least one Nurse.
- A Physician can treat many Patients; a Patient can be treated by many Physicians. A Diagnosis and TreatmentDate are recorded each time.

Draw ER diagram below:

37. Catering & Events — Draw the complete ER diagram for the following business rules.

Business Rules:

- The company manages Staff, Events, Menus, and Dishes.
- Each Staff member has an EmpID, Name, and Salary. A Staff member may supervise other Staff members.
- Each Staff member can hold multiple Skills.
- Events have an EventID, EventDate, EventLocation, and EventType.
- Each Event uses exactly one Menu. A Menu can be reused across many Events.
- A Menu is composed of many Dishes; a Dish can appear on many Menus.
- Each Dish has a DishID, DishName, and PrepTime.
- Staff members are assigned to Events. Each assignment records a Position, StartTime, and EndTime. A Staff member can work many Events; an Event requires many Staff.

Draw ER diagram below: